

Теория Параллелизма

Отчет

Решения уравнения
теплопроводности методом
разностной схемы

Выполнил: Быстрых Никита, группа 24940

1.06.2026

Цель: Исследовать производительность реализации на CPU и GPU, выполнить профилирование Nsight Systems, затем применить и оценить оптимизацию GPU-варианта.

Используемый компилятор: NVIDIA HPC SDK (nvcc++) 23.11-0 64-bit

Используемый профилировщик: NVIDIA Nsight system

Замеры производились с помощью библиотеки <chrono>

Выполнение на CPU

CPU-onecore

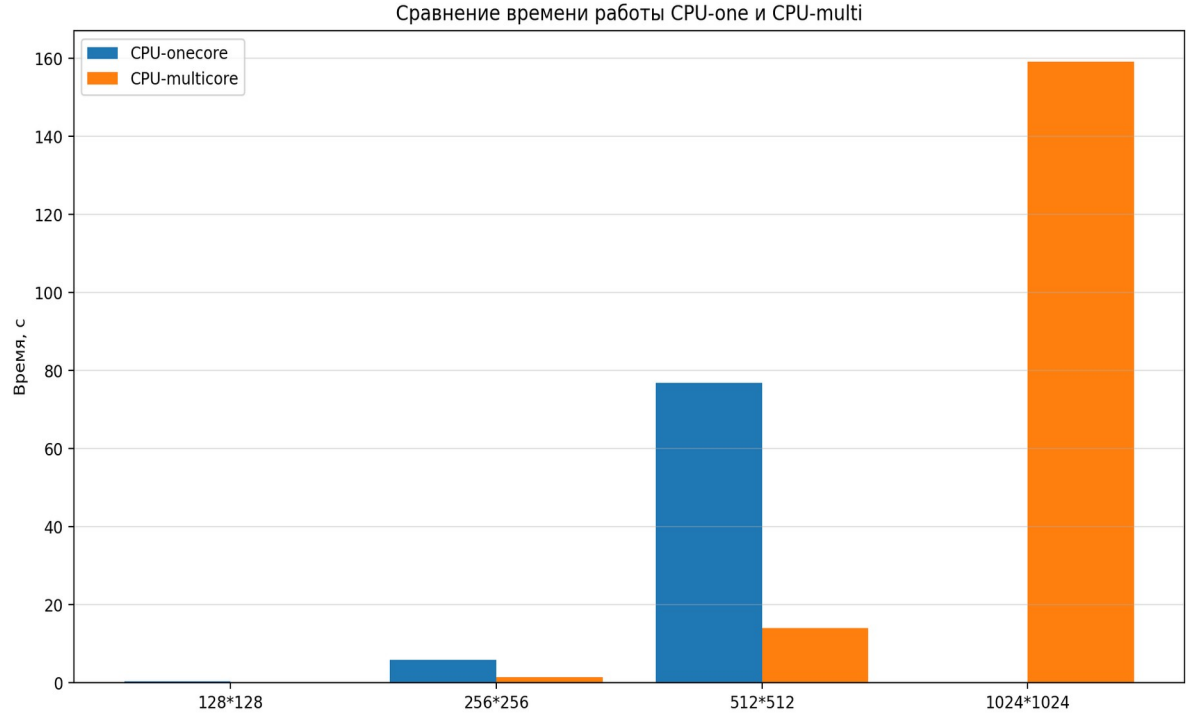
Размер сетки	Время выполнения (с)	Точность	Количество итераций
128x128	0.437138	9.9998e-7	30074
256x256	5.864524	9.9993e-7	102885
512x512	76.837177	9.9998e-7	339599

CPU-multicore

Размер сетки	Время выполнения (с)	Точность	Количество итераций
128x128	0.163239	9.9998e-7	30074
256x256	1.467721	9.9993e-7	102885

512x512	13.976909	9.9998e-7	339599
1024x1024	159.236667	1.3692e-6	1 000 000

Диаграмма сравнения CPU oncore vs multicore

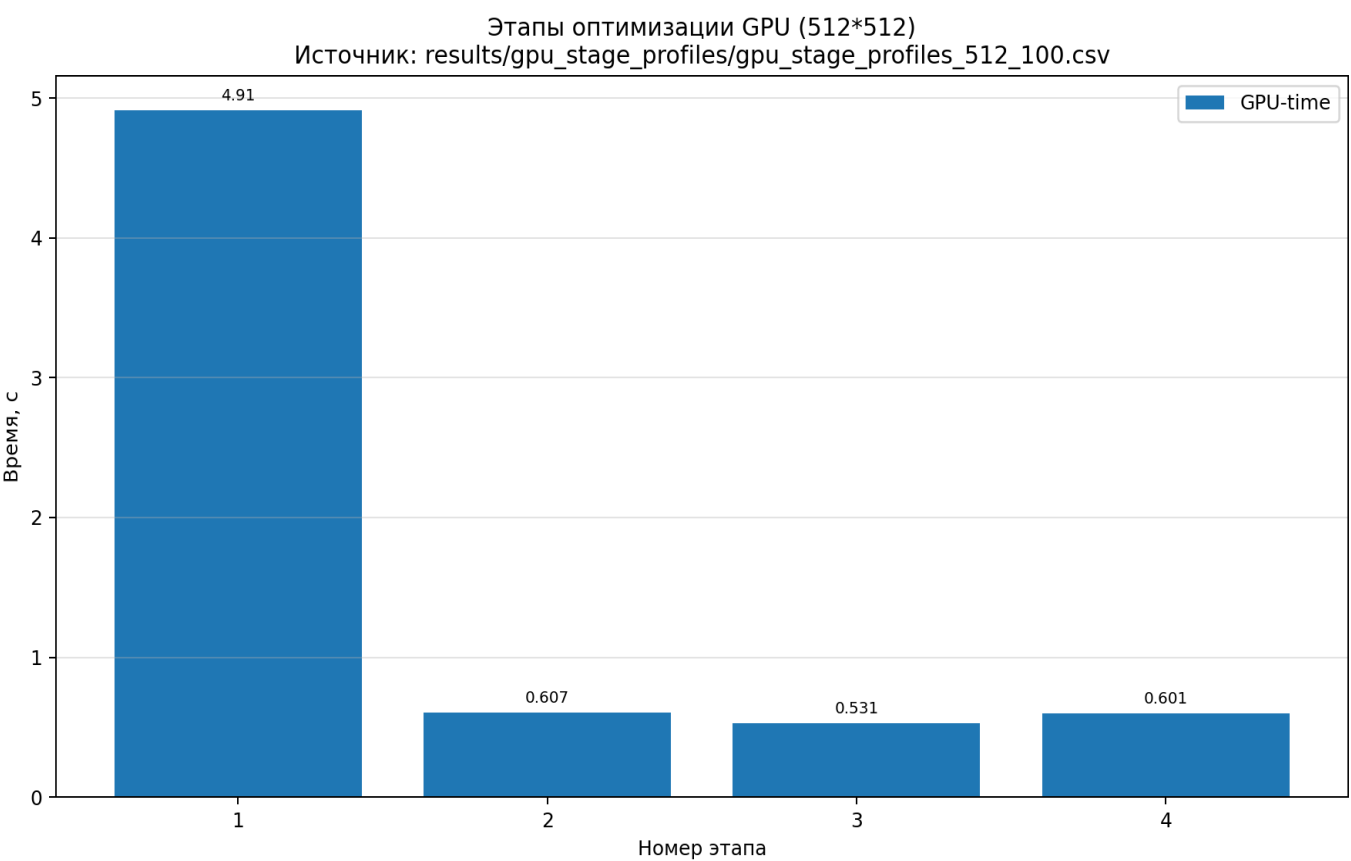


Выполнение на GPU

Этапы оптимизации на сетке 512x512

Этап No	Время выполнения	Точность	Максимальное количество итераций	Комментарии
1	4.914714	1e-6	100	Multicore — реализация, запущенная на GPU
2	0.374754	1e-6	100	Стартовый вариант OpenACC, неявные копирования между CPU и GPU
3	0.296437	1e-6	100	Добавлена область данных, матрицы остаются в памяти GPU

4	0.287966	1e-6	100	Реже вычисляется ошибка сходимости, меньше синхронизаций
---	----------	------	-----	---



Этап №1

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
48,0	4 343 163 451	100	43 431 634,0	43 062 267,0	43 055 701	52 744 288	1 812 935,0	Compute Construct@therm_gpu.cpp:1
48,0	4 341 189 519	100	43 411 895,0	43 044 555,0	43 038 572	52 687 471	1 808 069,0	Wait@therm_gpu.cpp:112
0,0	63 456 943	200	317 284,0	295 212,0	12 391	1 050 466	312 328,0	Enter Data@therm_gpu.cpp:109
0,0	50 310 707	300	167 702,0	208 792,0	5 694	473 272	121 418,0	Enqueue Upload@therm_gpu.cpp:109
0,0	39 640 264	200	198 201,0	237 561,0	21 318	540 267	172 741,0	Exit Data@therm_gpu.cpp:109
0,0	37 504 058	200	187 520,0	207 728,0	15 551	521 862	171 897,0	Enqueue Download@therm_gpu.cpp:12
0,0	11 102 195	200	55 511,0	58 011,0	2 031	138 042	51 845,0	Wait@therm_gpu.cpp:109
0,0	1 513 984	100	15 139,0	13 516,0	7 795	36 471	5 167,0	Enqueue Launch@therm_gpu.cpp:112
0,0	727 410	1	727 410,0	727 410,0	727 410	727 410	0,0	Device Init
0,0	632 968	200	3 164,0	2 489,0	1 175	16 542	2 314,0	Wait@therm_gpu.cpp:123
0,0	0	3	0,0	0,0	0	0	0,0	Alloc@therm_gpu.cpp:109
0,0	0	300	0,0	0,0	0	0	0,0	Create@therm_gpu.cpp:109
0,0	0	300	0,0	0,0	0	0	0,0	Delete@therm_gpu.cpp:123
0,0	0	3	0,0	0,0	0	0	0,0	Free@therm_gpu.cpp:123

Этап №2

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
28,0	64 475 109	200	322 375,0	334 916,0	12 046	1 270 624	312 657,0	Enter Data@therm_gpu.cpp:149
22,0	51 264 494	300	170 881,0	224 450,0	4 929	456 092	121 083,0	Enqueue Upload@therm_gpu.cpp:149
20,0	45 594 078	200	227 970,0	217 500,0	16 229	696 417	210 613,0	Exit Data@therm_gpu.cpp:149
19,0	43 941 098	200	219 705,0	199 111,0	11 856	685 801	208 667,0	Enqueue Download@therm_gpu.cpp:163
5,0	13 327 318	300	44 424,0	19 616,0	3 503	152 266	47 212,0	Wait@therm_gpu.cpp:149
1,0	4 013 166	100	40 131,0	38 943,0	36 883	79 197	4 763,0	Compute Construct@therm_gpu.cpp:149
0,0	1 770 618	200	8 853,0	8 085,0	5 431	33 974	3 390,0	Enqueue Launch@therm_gpu.cpp:149
0,0	728 651	1	728 651,0	728 651,0	728 651	728 651	0,0	Device Init
0,0	496 030	200	2 480,0	2 410,0	1 413	8 949	1 173,0	Wait@therm_gpu.cpp:163
0,0	0	3	0,0	0,0	0	0	0,0	Alloc@therm_gpu.cpp:149
0,0	0	300	0,0	0,0	0	0	0,0	Create@therm_gpu.cpp:149
0,0	0	300	0,0	0,0	0	0	0,0	Delete@therm_gpu.cpp:163
0,0	0	3	0,0	0,0	0	0	0,0	Free@therm_gpu.cpp:163

Этап №3

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
21,0	3 646 970	100	36 469,0	33 165,0	32 439	130 065	14 115,0	Compute Construct@therm_gpu.cpp:78
15,0	2 699 393	300	8 998,0	3 460,0	1 030	26 506	9 495,0	Wait@therm_gpu.cpp:78
10,0	1 739 645	1	1 739 645,0	1 739 645,0	1 739 645	1 739 645	0,0	Enter Data@therm_gpu.cpp:189
9,0	1 690 627	200	8 453,0	9 698,0	808	140 341	12 750,0	Exit Data@therm_gpu.cpp:78
8,0	1 457 372	1	1 457 372,0	1 457 372,0	1 457 372	1 457 372	0,0	Device Init
8,0	1 433 820	200	7 169,0	7 694,0	2 059	117 179	9 564,0	Enter Data@therm_gpu.cpp:78
7,0	1 205 309	200	6 026,0	4 180,0	3 635	92 568	8 012,0	Enqueue Launch@therm_gpu.cpp:78
6,0	1 087 124	100	10 871,0	8 736,0	8 053	63 460	8 236,0	Enqueue Download@therm_gpu.cpp:92
5,0	983 338	2	491 669,0	491 669,0	475 627	507 711	22 686,0	Enqueue Upload@therm_gpu.cpp:189
2,0	408 564	100	4 085,0	2 795,0	2 459	49 729	5 227,0	Enqueue Upload@therm_gpu.cpp:78
1,0	328 524	1	328 524,0	328 524,0	328 524	328 524	0,0	Update@therm_gpu.cpp:201
1,0	315 814	1	315 814,0	315 814,0	315 814	315 814	0,0	Enqueue Download@therm_gpu.cpp:201
0,0	150 496	100	1 505,0	1 214,0	1 050	7 613	1 014,0	Wait@therm_gpu.cpp:92
0,0	89 971	1	89 971,0	89 971,0	89 971	89 971	0,0	Wait@therm_gpu.cpp:189
0,0	22 969	1	22 969,0	22 969,0	22 969	22 969	0,0	Exit Data@therm_gpu.cpp:189
0,0	4 700	1	4 700,0	4 700,0	4 700	4 700	0,0	Wait@therm_gpu.cpp:201
0,0	0	2	0,0	0,0	0	0	0,0	Alloc@therm_gpu.cpp:189
0,0	0	1	0,0	0,0	0	0	0,0	Alloc@therm_gpu.cpp:78
0,0	0	2	0,0	0,0	0	0	0,0	Create@therm_gpu.cpp:189
0,0	0	100	0,0	0,0	0	0	0,0	Create@therm_gpu.cpp:78
0,0	0	2	0,0	0,0	0	0	0,0	Delete@therm_gpu.cpp:201
0,0	0	100	0,0	0,0	0	0	0,0	Delete@therm_gpu.cpp:92
0,0	0	3	0,0	0,0	0	0	0,0	Free@therm_gpu.cpp:201

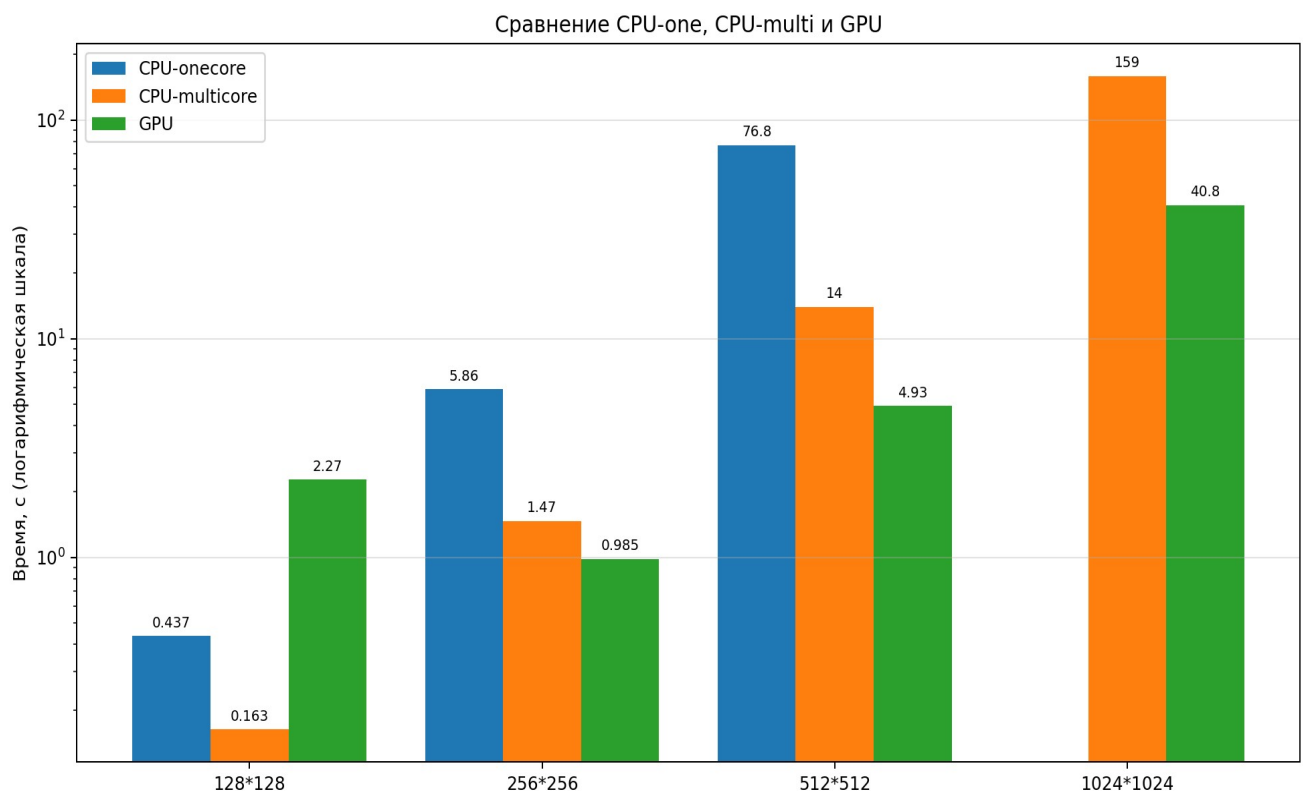
Этап №4

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
26,0	1 847 440	99	18 661,0	17 762,0	17 100	43 691	3 080,0	Compute Construct@therm_gpu.cpp:66
19,0	1 389 735	198	7 018,0	10 027,0	1 022	13 157	5 795,0	Wait@therm_gpu.cpp:66
11,0	824 969	1	824 969,0	824 969,0	824 969	824 969	0,0	Enter Data@therm_gpu.cpp:220
10,0	712 098	1	712 098,0	712 098,0	712 098	712 098	0,0	Device Init
6,0	435 400	2	217 700,0	217 700,0	217 281	218 119	592,0	Enqueue Upload@therm_gpu.cpp:220
6,0	428 622	99	4 329,0	3 670,0	3 467	24 539	2 516,0	Enqueue Launch@therm_gpu.cpp:66
4,0	336 902	1	336 902,0	336 902,0	336 902	336 902	0,0	Update@therm_gpu.cpp:242
4,0	326 552	1	326 552,0	326 552,0	326 552	326 552	0,0	Enqueue Download@therm_gpu.cpp:242
3,0	249 208	99	2 517,0	2 234,0	2 085	12 821	1 218,0	Enter Data@therm_gpu.cpp:66
1,0	125 075	99	1 263,0	943,0	856	20 651	2 110,0	Exit Data@therm_gpu.cpp:66
1,0	96 720	1	96 720,0	96 720,0	96 720	96 720	0,0	Wait@therm_gpu.cpp:220
0,0	47 401	2	23 700,0	23 700,0	1 186	46 215	31 840,0	Exit Data@therm_gpu.cpp:78
0,0	41 355	1	41 355,0	41 355,0	41 355	41 355	0,0	Compute Construct@therm_gpu.cpp:78
0,0	33 892	2	16 946,0	16 946,0	2 634	31 258	20 240,0	Enter Data@therm_gpu.cpp:78
0,0	28 458	1	28 458,0	28 458,0	28 458	28 458	0,0	Enqueue Download@therm_gpu.cpp:92
0,0	26 364	3	8 788,0	2 866,0	1 227	22 271	11 705,0	Wait@therm_gpu.cpp:78
0,0	10 820	2	5 410,0	5 410,0	4 893	5 927	731,0	Enqueue Launch@therm_gpu.cpp:78
0,0	8 481	1	8 481,0	8 481,0	8 481	8 481	0,0	Exit Data@therm_gpu.cpp:220
0,0	8 281	1	8 281,0	8 281,0	8 281	8 281	0,0	Enqueue Upload@therm_gpu.cpp:78
0,0	3 406	1	3 406,0	3 406,0	3 406	3 406	0,0	Wait@therm_gpu.cpp:242
0,0	1 834	1	1 834,0	1 834,0	1 834	1 834	0,0	Wait@therm_gpu.cpp:92
0,0	0	2	0,0	0,0	0	0	0,0	Alloc@therm_gpu.cpp:220
0,0	0	1	0,0	0,0	0	0	0,0	Alloc@therm_gpu.cpp:78
0,0	0	2	0,0	0,0	0	0	0,0	Create@therm_gpu.cpp:220
0,0	0	1	0,0	0,0	0	0	0,0	Create@therm_gpu.cpp:78
0,0	0	2	0,0	0,0	0	0	0,0	Delete@therm_gpu.cpp:242
0,0	0	1	0,0	0,0	0	0	0,0	Delete@therm_gpu.cpp:92
0,0	0	3	0,0	0,0	0	0	0,0	Free@therm_gpu.cpp:242

GPU - Оптимизированный вариант

Размер сетки	Время выполнения (с)	Точность	Количество итераций
128x128	2.269503	9.9205e-07	30100
256x256	0.984548	9.9879e-07	102900
512x512	4.932967	9.9996e-07	339600
1024x1024	40.830705	1.3692e-6	1000000

Диаграмма сравнения времени работы CPU-one, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток



Вывод:

На малых сетках ускорение от GPU выражено слабо, так как значительную часть времени занимают накладные расходы: инициализация ускорителя, запуск OpenACC-конструкций, синхронизация и передача данных между CPU и GPU. На больших сетках доля полезных вычислений возрастает, поэтому GPU используется эффективнее.

На первом этапе была запущена multicore-реализация на GPU. Такой вариант оказался неэффективным: компилятор не смог полноценно распараллелить вычислительный цикл, а время выполнения составило около 4.91 с.

На втором этапе была добавлена явная параллельная область OpenACC. Это позволило корректнее перенести вычисления на GPU и сократить время до 0.37 с.

На третьем этапе была добавлена область данных OpenACC, благодаря чему матрицы оставались в памяти GPU в течение всего итерационного процесса. Это уменьшило лишние передачи данных между CPU и GPU, а время снизилось до 0.29 с.

На четвертом этапе ошибка сходимости вычислялась реже, чтобы уменьшить число синхронизаций. Однако на тесте в 100 итераций этот вариант не дал значительного выигрыша и показал время около 0.28 с.

Основными ограничителями производительности были лишние пересылки данных, синхронизации и накладные расходы OpenACC. Лучшим вариантом по результатам профилирования стал четвёртый этап, так как он сохраняет исходный алгоритм Якоби и показывает минимальное время выполнения.